

UniMate AI

Software Requirements Specification (SRS)

Version 1.0

1. Introduction

1.1 Project Name

UniMate AI – Intelligent University Study Companion

1.2 Purpose

UniMate AI is an AI-powered web application designed for university students from any university, country, degree program, or academic discipline.

The platform allows students to organize their courses and upload academic materials such as PDFs, lecture slides, notes, and past examination papers.

UniMate AI then transforms these materials into an interactive study environment where students can:

- Ask questions about their own materials.
- Generate summaries.
- Generate quizzes.
- Generate flashcards.
- Identify weak topics.
- Track learning progress.
- Prepare for examinations.
- Organize courses by semester.
- Receive source-based AI answers.

The system combines document management, artificial intelligence, Retrieval-Augmented Generation (RAG), and student progress tracking.

2. Problem Statement

University students commonly experience the following problems:

- Lecture materials are scattered across WhatsApp, email, Google Drive, LMS platforms, and personal devices.
- Students receive large PDFs and presentations without knowing which information is most important.
- Students sometimes struggle to understand the lecturer's explanation.
- Students do not always know whether they truly understand a topic.
- Exam preparation frequently begins too late.
- Students repeatedly reread entire documents instead of focusing on weak areas.
- Past examination papers and notes are poorly organized.
- Generic AI chatbots may answer questions without using the student's actual course materials.
- Students use separate applications for notes, quizzes, flashcards, planning, and AI assistance.

UniMate AI solves these problems by providing one centralized intelligent study workspace.

3. Product Vision

UniMate AI should become a personal academic workspace for every university student.

Instead of asking:

"What should I study?"

the platform should eventually be able to tell the student:

"You understand Topics A and B well, but your recent quiz results show weakness in Topic C. Your examination is in five days, so today you should focus on Topic C and complete this 15-question quiz."

4. Target Users

Primary User

University Student.

The system must work for students regardless of:

- University.
- Country.

- Degree.
- Faculty.
- Academic year.
- Language.
- Course.

Examples include students studying:

- Software Engineering.
 - Medicine.
 - Nursing.
 - Business.
 - Law.
 - Engineering.
 - Accounting.
 - Economics.
 - Computer Science.
 - Education.
 - Nutrition.
-

5. User Roles

5.1 Student

Students can:

- Register and log in.
- Manage their profile.
- Create semesters.
- Create courses.
- Upload course materials.
- Ask AI questions.
- Generate summaries.
- Generate flashcards.
- Generate quizzes.
- Attempt quizzes.
- View results.
- Track course progress.
- Manage notes.
- Delete their files and data.

5.2 Administrator

Administrators can:

- Manage users.
- View system statistics.
- Suspend abusive accounts.
- Review reported issues.
- Manage system limits.
- View application usage.
- Manage supported file types.
- Monitor AI-related failures.

Admin functionality is not required for the earliest prototype but the architecture must allow it later.

6. MVP Scope

The first production-ready MVP must contain:

1. Authentication.
 2. User onboarding.
 3. Dashboard.
 4. Semester management.
 5. Course management.
 6. File upload.
 7. Document processing.
 8. AI chat with course materials.
 9. Source citations.
 10. AI summaries.
 11. Flashcard generation.
 12. Quiz generation.
 13. Quiz attempts.
 14. Progress tracking.
 15. Notes.
 16. Search.
 17. Settings/profile.
 18. Responsive UI.
 19. Dark/light mode.
-

7. Out of Scope for MVP

The following features should NOT be implemented initially:

- Live video classes.
- University LMS integration.
- Teacher accounts.
- Payments.
- Social networking.
- Group chat.
- Marketplace.
- Native Android/iOS applications.
- Automatic university timetable scraping.
- Audio podcast generation.
- Voice conversations.
- Collaborative document editing.

These may be implemented in future versions.

8. User Onboarding

When a new user registers, the system should ask for:

- Full name.
- University name.
- Country.
- Degree/program.
- Current academic year.
- Preferred study language.

Optional information:

- Faculty.
- Expected graduation year.

The application must allow students to skip non-essential onboarding fields.

9. Authentication Requirements

The system must support:

- Email registration.
- Email/password login.

- Logout.
- Password hashing.
- Password reset architecture.
- Secure authentication tokens or sessions.
- Protected routes.
- Authentication persistence.

Future support may include:

- Google OAuth.
 - Microsoft OAuth.
 - University SSO.
-

10. Dashboard

After login, the student sees a dashboard.

The dashboard should contain:

- Welcome message.
- Current semester.
- Courses.
- Recent study activity.
- Upcoming exams if available.
- Recent documents.
- Recent quizzes.
- Course progress.
- Continue studying section.
- Quick upload button.
- Quick course creation button.

Example:

Welcome back, Alex.

Courses:

Database Administration — 72% Computer Networks — 64% Java Programming — 51% Operating Systems — 83%

Recent activity:

- Completed DB Quiz 2.
- Added Networking Lecture 5.
- Generated 15 flashcards.

11. Semester Management

Students must be able to:

- Create semester.
- Rename semester.
- Archive semester.
- Delete semester.
- Select active semester.

Semester fields:

- ID.
- User ID.
- Name.
- Academic year.
- Start date.
- End date.
- Status.
- Created timestamp.
- Updated timestamp.

Example:

Semester 1 2026–2027

12. Course Management

Students must be able to:

- Create course.
- Edit course.
- Delete course.
- Archive course.
- Assign course to semester.
- Select a color/icon.
- Add optional course code.

Course fields:

- ID.
- User ID.
- Semester ID.
- Course name.

- Course code.
- Description.
- Color.
- Created timestamp.
- Updated timestamp.

Example:

Course Name: Database Administration

Course Code: CSC302

13. Course Workspace

Each course should have an independent study workspace.

Sections:

- Overview.
- Sources.
- AI Chat.
- Notes.
- Flashcards.
- Quizzes.
- Progress.

The student must only receive AI answers from files belonging to the selected course unless explicitly using general AI mode.

14. Source Management

Students must be able to upload academic materials.

Supported MVP formats:

- PDF.
- TXT.
- DOCX where technically practical.
- PPTX where technically practical.

PDF must be prioritized.

Each source stores:

- ID.
- Course ID.
- User ID.
- File name.
- File type.
- File size.
- Storage URL/path.
- Processing status.
- Extracted text.
- Upload date.
- Number of pages where available.

Processing status:

- Uploading.
- Processing.
- Ready.
- Failed.

Students must be able to:

- Upload.
- View.
- Rename.
- Delete.
- Search sources.

15. Document Processing

After upload:

1. Validate file.
2. Save file.
3. Extract text.
4. Clean extracted text.
5. Split text into chunks.
6. Generate embeddings.
7. Store embeddings.
8. Mark document as ready.

Every chunk should preserve metadata such as:

- Source ID.
- File name.

- Page number when available.
- Chunk index.
- Course ID.
- User ID.

This metadata will be used for citations.

16. RAG System

The application should use Retrieval-Augmented Generation.

Workflow:

Student question

→ Generate query embedding

→ Search relevant document chunks

→ Retrieve top relevant chunks

→ Send retrieved context + question to AI model

→ Generate response

→ Return response with citations

The AI should prioritize uploaded course material.

It must avoid claiming unsupported information comes from the student's documents.

When the documents do not contain enough information, the assistant should clearly say so.

Example:

"The uploaded material does not provide enough information to answer this question."

17. AI Chat

Students can chat with the selected course.

Example questions:

- Explain normalization.
- What does Lecture 3 say about ACID properties?
- Compare TCP and UDP using my notes.
- Explain this topic in simpler English.
- Give me an exam-style explanation.
- What are the most important topics in these files?

Chat requirements:

- Conversation history.
 - Markdown rendering.
 - Loading states.
 - Error handling.
 - Source references.
 - File/page citations when possible.
 - Code formatting.
 - Mathematical formatting where practical.
-

18. AI Answer Modes

The user may choose:

Explain

Explain a topic clearly.

Simplify

Explain using easier language.

Summarize

Create a concise summary.

Study

Teach step by step.

Exam Prep

Focus on examinable concepts.

These modes affect prompting but use the same underlying RAG system.

19. Source Citations

AI responses based on uploaded documents should show citations.

Example:

ACID consists of Atomicity, Consistency, Isolation, and Durability.

Sources: Lecture-03.pdf — Page 14

Clicking a citation should identify the relevant source.

20. Summary Generation

Students must be able to generate summaries from:

- One document.
- Multiple selected documents.
- Entire course.

Summary styles:

- Short.
- Detailed.
- Exam revision.
- Key points.

Generated summaries should be saved.

Summary fields:

- ID.
 - User ID.
 - Course ID.
 - Source IDs.
 - Title.
 - Content.
 - Type.
 - Created date.
-

21. Flashcards

Students must be able to generate flashcards from their course material.

Each flashcard has:

- Question/front.
- Answer/back.
- Topic.
- Course ID.
- Source ID if available.

Functions:

- Generate flashcards.
- Flip card.
- Next card.
- Previous card.
- Mark known.
- Mark difficult.
- Regenerate cards.
- Delete card.

Future versions may implement spaced repetition.

22. Quiz System

Students must be able to create AI-generated quizzes.

Quiz configuration:

- Course.
- Source(s).
- Number of questions.
- Difficulty.
- Question type.

MVP question types:

- Multiple choice.
- True/False.

Future question types:

- Short answer.

- Essay.
- Coding.

Difficulty:

- Easy.
- Medium.
- Hard.
- Mixed.

Quiz fields:

- ID.
- Course ID.
- User ID.
- Title.
- Difficulty.
- Question count.
- Created date.

Question fields:

- ID.
- Quiz ID.
- Question.
- Options.
- Correct answer.
- Explanation.
- Topic.
- Source reference where available.

23. Quiz Attempt

The student should:

1. Start quiz.
2. Answer questions.
3. Submit quiz.
4. Receive score.
5. Review answers.
6. Read explanations.
7. View weak topics.

Attempt fields:

- ID.

- User ID.
 - Quiz ID.
 - Score.
 - Correct count.
 - Incorrect count.
 - Started timestamp.
 - Completed timestamp.
-

24. Weak Topic Detection

The platform should use quiz history to detect weak topics.

Example:

Weak Areas

Normalization — 45% Transactions — 62% Backup & Restore — 84%

The algorithm can initially be simple:

Topic mastery = correct answers / total topic questions × 100.

This can become more sophisticated later.

25. Progress Tracking

Each course should show:

- Number of documents.
- Number of quizzes completed.
- Average quiz score.
- Flashcards reviewed.
- Weak topics.
- Recent activity.

Course progress should not pretend to measure complete academic knowledge.

It should be presented as platform-based study progress.

26. Notes

Students must be able to:

- Create note.
- Edit note.
- Delete note.
- Associate note with course.
- Search notes.

Optional:

- Markdown editor.
- Rich-text editor.

Notes must autosave where practical.

27. Search

Students should be able to search:

- Courses.
- Documents.
- Notes.

Future:

- Semantic search inside documents.
-

28. Profile

Profile fields:

- Full name.
- Email.
- University.
- Program.
- Country.
- Study language.
- Avatar.
- Academic year.

Users must be able to update editable profile data.

29. Settings

Settings should include:

- Profile.
- Theme.
- Preferred AI response language.
- Delete account.
- Data management.

Future:

- Subscription.
 - AI model preferences.
 - Notification preferences.
-

30. Language Support

The application architecture should support multiple languages.

Initial UI may be English.

The AI assistant should be able to answer in the user's selected language where supported.

Examples:

- English.
 - Arabic.
 - French.
-

31. UI/UX Requirements

The application should have a modern academic SaaS design.

Design inspiration may be taken from:

- NotebookLM.
- Notion.
- Linear.
- ChatGPT.
- modern learning management interfaces.

Do NOT directly copy protected branding or layouts.

Requirements:

- Clean.
 - Minimal.
 - Professional.
 - Mobile responsive.
 - Tablet responsive.
 - Desktop optimized.
 - Accessible navigation.
 - Consistent spacing.
 - Empty states.
 - Skeleton loaders.
 - Toast notifications.
 - Confirmation dialogs.
-

32. Main Pages

Public:

- Landing Page.
- Login.
- Register.
- Forgot Password.

Private:

- Onboarding.
 - Dashboard.
 - Semesters.
 - Course Workspace.
 - Sources.
 - Chat.
 - Notes.
 - Flashcards.
 - Quizzes.
 - Quiz Attempt.
 - Quiz Results.
 - Progress.
 - Profile.
 - Settings.
-

33. Landing Page

Landing page sections:

- Navbar.
- Hero.
- Problem.
- Features.
- How it works.
- AI study workflow.
- Screenshots/demo area.
- FAQ.
- CTA.
- Footer.

Example hero message:

Turn your university materials into your personal AI tutor.

Upload lectures, ask questions, generate quizzes, create flashcards, and prepare smarter for exams.

34. Suggested Technology Stack

Frontend

- React.
- Vite.
- TypeScript.
- React Router.
- Tailwind CSS.
- shadcn/ui where useful.
- TanStack Query.
- React Hook Form.
- Zod.

Backend

- Node.js.
- Express.
- TypeScript.

Database

- PostgreSQL.

ORM

- Prisma.

Vector Search

Recommended MVP:

- PostgreSQL + pgvector.

Alternative:

- Pinecone.
- Qdrant.
- Weaviate.

File Storage

Development:

- Local uploads.

Production:

- Cloudinary.
- AWS S3.
- Supabase Storage or equivalent object storage.

AI

Provider abstraction should be created.

Possible providers:

- OpenAI.
- Gemini.
- other compatible models.

Never hard-code the application to one AI provider.

35. Suggested Architecture

Frontend

↓ REST API

Express Backend

↓ Authentication Service

↓ Course Service

↓ Document Service

↓ AI/RAG Service

↓ PostgreSQL + pgvector

↓ Object Storage

↓ External AI Provider

36. API Structure

Suggested API prefix:

/api/v1

Authentication:

POST /auth/register

POST /auth/login

POST /auth/logout

GET /auth/me

Courses:

GET /courses

POST /courses

GET /courses/:id

PATCH /courses/:id

DELETE /courses/:id

Semesters:

GET /semesters

POST /semesters

PATCH /semesters/:id

DELETE /semesters/:id

Sources:

POST /courses/:courseId/sources

GET /courses/:courseId/sources

GET /sources/:id

DELETE /sources/:id

Chat:

POST /courses/:courseId/chat

GET /courses/:courseId/conversations

GET /conversations/:id/messages

Summaries:

POST /courses/:courseId/summaries

GET /courses/:courseId/summaries

Flashcards:

POST /courses/:courseId/flashcards/generate

GET /courses/:courseId/flashcards

Quizzes:

POST /courses/:courseId/quizzes

GET /courses/:courseId/quizzes

GET /quizzes/:id

POST /quizzes/:id/attempts

GET /quizzes/:id/results

Progress:

GET /courses/:courseId/progress

Notes:

GET /courses/:courseId/notes

POST /courses/:courseId/notes

PATCH /notes/:id

DELETE /notes/:id

37. Database Entities

Core entities:

User

Semester

Course

Source

DocumentChunk

Conversation

Message

Citation

Note

Summary

Flashcard

Quiz

Question

QuizAttempt

QuizAnswer

TopicMastery

Activity

38. Proposed Relationships

User has many Semesters.

User has many Courses.

Semester has many Courses.

Course has many Sources.

Source has many DocumentChunks.

Course has many Conversations.

Conversation has many Messages.

Message may have many Citations.

Course has many Notes.

Course has many Flashcards.

Course has many Quizzes.

Quiz has many Questions.

Quiz has many Attempts.

User has many Attempts.

Course has many TopicMastery records.

39. Security Requirements

The application must:

- Hash passwords securely.
- Validate all inputs.
- Protect authenticated endpoints.
- Prevent access to another user's data.
- Validate uploaded files.
- Limit upload size.
- Prevent dangerous executable uploads.
- Use environment variables.
- Never expose AI/API secrets to frontend.
- Use secure headers.
- Implement rate limiting.
- Sanitize appropriate inputs.
- Handle CORS correctly.
- Protect against common OWASP risks.

Every resource query must verify ownership.

Example:

User A must never access User B's course by guessing its ID.

40. AI Security

Uploaded content is untrusted input.

The system must protect against prompt injection contained inside uploaded documents.

The RAG prompt should clearly distinguish:

- System instructions.
- User request.
- Retrieved document content.

Retrieved documents must not be allowed to override system behavior.

41. Privacy

User documents are private by default.

The platform must not expose one user's files to another.

Deletion of a source should remove:

- Stored document.
- Extracted content.
- Related embeddings/chunks.

Account deletion should eventually support full personal-data deletion.

42. Performance Requirements

Target MVP:

- Normal API requests: approximately < 1 second where practical.
- AI responses may take longer and should stream where practical.
- UI must show loading indicators.
- Large document processing may happen asynchronously.

The backend should not block the main server thread with heavy file processing.

43. Error Handling

All API errors should use a consistent format.

Example:

```
{ "success": false, "error": { "code": "COURSE_NOT_FOUND", "message": "Course not found." } }
```

Frontend must display user-friendly messages.

44. Logging

Backend should log:

- Application errors.

- AI errors.
- Document processing failures.
- Authentication failures.
- Unexpected exceptions.

Never log:

- Passwords.
 - Auth tokens.
 - Complete sensitive user documents unnecessarily.
-

45. Testing Requirements

Backend:

- Unit tests.
- Integration tests.
- API tests.

Frontend:

- Component tests where valuable.
- Core flow tests.

Critical flows:

- Registration.
 - Login.
 - Course creation.
 - File upload.
 - Chat.
 - Quiz generation.
 - Quiz submission.
 - Authorization.
-

46. Development Requirements

Use:

- ESLint.
- Prettier.
- TypeScript strict mode where practical.
- Git.
- Environment variables.

- Clear README.

Do not:

- Hardcode secrets.
 - Create giant components.
 - Put business logic directly inside route files.
 - Duplicate code unnecessarily.
-

47. Suggested Repository Structure

unimate-ai/

apps/

web/

server/

packages/

shared/

docs/

docker/

README.md

Alternative simpler structure:

client/

server/

README.md

For the MVP, a monorepo is recommended.

48. Backend Structure

src/

config/

controllers/

routes/

services/

repositories/

middleware/

schemas/

lib/

ai/

rag/

jobs/

utils/

types/

app.ts

server.ts

49. Frontend Structure

src/

components/

pages/

features/

hooks/

services/

lib/

layouts/

routes/

types/

assets/

App.tsx

main.tsx

50. Environment Variables

Examples:

DATABASE_URL

JWT_SECRET

AI_API_KEY

AI_PROVIDER

AI_MODEL

EMBEDDING_MODEL

STORAGE_PROVIDER

STORAGE_BUCKET

CLIENT_URL

PORT

NODE_ENV

No real keys must be committed.

51. Development Phases

Phase 1

Project foundation.

- Monorepo.
- Frontend.
- Backend.
- Database.
- Prisma.
- Environment setup.

Phase 2

Authentication.

Phase 3

Semester and course management.

Phase 4

Source upload.

Phase 5

PDF text extraction.

Phase 6

Embedding pipeline.

Phase 7

RAG chat.

Phase 8

Summaries.

Phase 9

Flashcards.

Phase 10

Quizzes.

Phase 11

Progress tracking.

Phase 12

Testing and polish.

Phase 13

Deployment.

52. MVP Acceptance Criteria

The MVP is considered successful when a new student can:

1. Register.
 2. Complete onboarding.
 3. Create a semester.
 4. Create a course.
 5. Upload a PDF.
 6. Wait for the PDF to process.
 7. Ask a question about the PDF.
 8. Receive an answer grounded in the PDF.
 9. See a source citation.
 10. Generate a summary.
 11. Generate flashcards.
 12. Generate an MCQ quiz.
 13. Complete the quiz.
 14. See the result.
 15. See weak topics.
 16. Return later and continue studying.
-

53. Future Roadmap

Possible future features:

- Study planner.
- Examination countdown.
- AI-generated study schedules.
- Audio summaries.
- Podcast generation.
- Voice tutor.
- Collaborative notebooks.
- Study groups.
- Shared flashcards.
- Shared course resources.
- Past-paper library.
- GPA calculator.
- Calendar.
- Reminders.
- Push notifications.
- Native mobile app.
- Offline mode.
- OCR.
- Recorded lecture transcription.
- YouTube source import.
- Website source import.
- Google Drive integration.
- OneDrive integration.
- LMS integration.
- Instructor accounts.
- University partnerships.

54. Product Principle

UniMate AI should not be designed to simply give students answers.

It should help students:

Understand → Practice → Discover weaknesses → Improve → Prepare.

The product should behave like a study companion, not an academic cheating tool.

55. Final Product Statement

UniMate AI is an AI-powered university study workspace that transforms a student's own academic materials into interactive learning experiences including source-grounded conversations, summaries, flashcards, quizzes, and personalized progress insights.

The platform is university-independent, discipline-independent, and designed to eventually serve university students worldwide.